Matrix Factorization and Genre-Based Recommender Systems on MovieLens-32M

Joel Gedeon Asa Mapa [1]

Abstract

Training a recommender system on tens of millions of ratings requires both a suitable model and memory-efficient data structures. In this paper, we build and test matrix factorization models trained with Alternating Least Squares (ALS) on the MovieLens-32M dataset (32,000,204 ratings from 200,948 users and 84,432 movies) and a 5-million-rating subset. To store the ratings in memory, we use two Compressed Sparse Row (CSR) matrices indexed by user and by movie, which take 513 MB of RAM and allow fast rating lookups. By running the ALS updates across CPU threads and solving the small linear systems in-place with Cholesky decomposition, our training loop runs about $70\times$ faster than batched array solvers. We examine the rating distribution and power-law behavior of users and movies, and compare a bias-only baseline against full matrix factorization across regularization values and latent dimensions K. We also plot 2D movie vectors, generate recommendations for a new user who likes *The Lord of the Rings*, and use vector lengths to find the most and least polarizing movies. Finally, adding movie genre priors helps predictions for new and rarely rated movies. Code: https://github.com/joelinator/movielens-32m-recommender.

1. Introduction

Online platforms such as streaming services and online stores use recommender systems to help users find items they might like among thousands of choices (Koren et al., 2009; Harper & Konstan, 2015). A common approach is collaborative filtering, which predicts what a user will like from past ratings across users, without needing descriptions of the items.

On large datasets like MovieLens-32M (32,000,204 ratings across 200,948 users and 84,432 movies), storing ratings in a standard 2D table is not practical: a $200{,}948 \times 84{,}432$ float32 array would take 67.8 GB of RAM, even though 99.81% of the table is empty because each user watches only a small number of movies.

Matrix factorization handles this sparsity by representing each user m and movie n with a vector of K numbers, $u_m, v_n \in \mathbb{R}^K$, called latent trait vectors (Mnih & Salakhutdinov, 2007; Koren et al., 2009), and predicting ratings as $u_m^\top v_n$ plus user and movie biases. Two algorithms are commonly used to learn these parameters: Stochastic Gradient Descent (SGD) and Alternating Least Squares (ALS) (Zhou et al., 2008). ALS works well on large datasets because: (1) it has no learning rate to tune, since each step has a closed-form formula; (2) the regularized loss goes down at every step; and (3) fixing movie vectors lets all users be updated in parallel, and vice versa for movies.

1.1. Problem Statement and Structure of the Paper

Our goal is to predict unseen movie ratings on MovieLens-32M while understanding what the model learns and where it struggles:

1. **Data Representation and Exploration (Section 2):** We store all 32M ratings in two CSR matrices ($\mathcal{S}_{\text{user}}$ and $\mathcal{S}_{\text{movie}}$) for fast user and movie lookups, and examine the rating histogram and user/movie degree distributions.

2. **Models and ALS Optimization (Section 3):** We present all three models and derive their ALS updates: (i) the bias-only baseline (Section 3.1), (ii) matrix factorization with trait vectors U and V (Section 3.2), and (iii) genre-regularized matrix factorization (Section 3.3), along with our parallel Cholesky solver (Section 3.4).

[1]African Institute for Mathematical Sciences (AIMS) South Africa, 6 Melrose Road, Muizenberg 7975, Cape Town, South Africa. Correspondence to: Joel Gedeon Asa Mapa <joel@aims.ac.za>.



3. **Experimental Results (Section 4):** We test all models on MovieLens-5M and MovieLens-32M, looking at loss/RMSE convergence, hyperparameter tuning (λ, γ, τ), rank K scaling and overfitting on users with few ratings, runtime speedups, 2D movie and genre maps, *Lord of the Rings* dummy-user recommendations, and most/least polarizing movies.

2. Data Representation and Exploratory Analysis

2.1. Dual Sparse Matrix Structure

Let M be the number of users and N be the number of movies. The dataset consists of observed ratings $r_{m,n} \in \{0.5, 1.0, \ldots, 5.0\}$ for user-movie pairs $(m, n) \in \Omega$. For each user m, we write $\Omega_m = \{n : (m, n) \in \Omega\}$ for the set of movies rated by user m. Similarly, for each movie n, we write $\Omega^n = \{m : (m, n) \in \Omega\}$ for the set of users who rated movie n, with total ratings $|\Omega| = \sum_{m=1}^{M} |\Omega_m| = \sum_{n=1}^{N} |\Omega^n|$.

During ALS training, updating user parameters requires looping over all movies rated by each user (Ω_m), while updating movie parameters requires looping over all users who rated each movie (Ω^n). Storing the data only by user would require scanning the whole dataset or jumping randomly in memory during movie updates, while nested lists or hash tables use too much memory. To make both half-steps fast, we store the ratings in two Compressed Sparse Row (CSR) structures (Section B):

- **User-Indexed CSR ($\mathcal{S}_{\text{user}}$):** Uses three flat arrays: an offset array $p^{(u)}$ of length $M + 1$, a movie-index array of length $|\Omega|$, and a rating array of length $|\Omega|$. User m's rated movie IDs and ratings sit side-by-side in memory between indices $p_m^{(u)}$ and $p_{m+1}^{(u)}$.
- **Movie-Indexed CSR ($\mathcal{S}_{\text{movie}}$):** Uses the transposed layout by movie: an offset array $p^{(v)}$ of length $N + 1$, a user-index array of length $|\Omega|$, and a rating array of length $|\Omega|$. Movie n's ratings sit side-by-side between indices $p_n^{(v)}$ and $p_{n+1}^{(v)}$.

Using 32-bit integers for indices/offsets and 32-bit floats for ratings, both CSR views take $16|\Omega| + 4(M + N + 2)$ bytes. For all 32,000,204 ratings in MovieLens-32M (Table 1), both structures together take 513.1 MB of RAM (132× smaller than a dense table) and load in 4.23 s, allowing the CPU to read ratings in order from memory.

2.2. Rating Distribution and Power-Law Behavior

Figure 1(a) shows the distribution of star ratings from 0.5 to 5.0 on MovieLens-32M and 5M, which match within 0.02% on every bar:

Table 1. Summary statistics of the MovieLens-5M subset and the full MovieLens-32M dataset.

Statistic	ML-5M	ML-32M
Users (M) / Movies (N)	31,205 / 48,708	200,948 / 84,432
Total Ratings ($\|\Omega\|$)	5,000,000	32,000,204
Train / Test Ratings	4.00M / 1.00M	25.60M / 6.40M
Matrix Density	0.3290%	0.1886%
Mean Rating μ / Std. σ	3.540 / 1.060	3.540 / 1.059
User Deg. Min/Med/Avg/Max	16/73/160/9,577	20/73/159/33,332
Movie Deg. Min/Med/Avg/Max	1/3/103/16,041	1/5/379/102,929
Power-Law Slopes $\hat{\alpha}_{\mathcal{U}}$ / $\hat{\alpha}_{\mathcal{I}}$	1.90 / 1.51	1.90 / 1.48

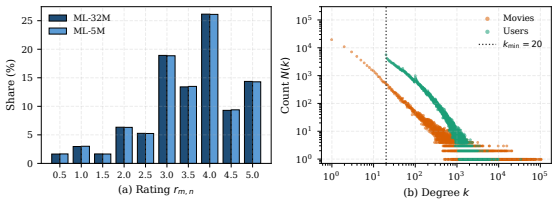


Figure 1. Exploratory plots of the MovieLens-32M and 5M datasets. **(a)** Percentage of ratings at each star value from 0.5 to 5.0, peaking at 4.0 stars with lower counts on half-star values. **(b)** Log-log plot of movie and user degree distributions on MovieLens-32M, showing power-law tails and the user cutoff at $k_{\min} = 20$.

1. **Users Tend to Give High Ratings:** The mean rating is $\mu = 3.540$ ($\sigma = 1.059$), well above the 2.75 midpoint. The most common score is 4.0 stars (26.15%, or 8,367,654 ratings), followed by 3.0 stars (18.92%) and 5.0 stars (14.36%), while low ratings ($r_{m,n} \in [0.5, 1.5]$) make up only 6.26% because users mostly watch movies they expect to like (Koren et al., 2009).
2. **Whole Stars Are Used More Than Half Stars:** Every half-star rating $(0.5, 1.5, 2.5, 3.5, 4.5)$ appears much less often than adjacent whole stars (3.5 is 13.41% vs. 18.92% for 3.0 and 26.15% for 4.0), both because MovieLens added half-star ratings in 2003 and because users naturally prefer round numbers.

Figure 1(b) plots the degree frequencies on a log-log scale. Both curves follow a straight line over several orders of magnitude, fitting a power law $P(k) \propto k^{-\alpha}$ (Clauset et al., 2009) with maximum-likelihood exponent $\hat{\alpha} = 1 + |\mathcal{T}|(\sum_{i \in \mathcal{T}} \ln \frac{k_i}{k_{\text{tail}} - 0.5})^{-1}$ ($k_{\text{tail}} = 50$), but differ at low counts:

- **Long Tail of Unpopular Movies:** Movie rating counts span $k = 1$ to $k_{\max} = 102{,}929$ (*The Shawshank Redemption (1994)*) with slope $\hat{\alpha}_{\mathcal{I}} = 1.48$. Out of 84,432 movies, 16,895 have 1 rating and the median is 5.0 (mean 379.0), so most movies have very few ratings.

- **Cutoff at $k_{\min} = 20$ for Users:** The user curve stops at $k_{\min} = 20$ on MovieLens-32M (median 73.0, mean 159.2, slope $\hat{\alpha}_{\mathcal{U}} = 1.90$) because MovieLens excluded users with fewer than 20 ratings (Harper & Konstan, 2015). Thus every user has at least 16 training ratings, whereas thousands of movies have $|\Omega^n_{\text{train}}| \in \{0, 1, 2\}$, making cold-start harder on the movie side (Section 3.3).

3. Models and Optimization

In this section, we present all three models developed in this paper and derive their ALS update rules: (1) the baseline bias-only model, (2) matrix factorization with user and movie trait vectors U and V, and (3) genre-regularized matrix factorization, followed by our parallel in-place Cholesky implementation.

3.1. Bias-Only Model

Before learning latent vectors, we build a baseline model that captures general rating habits: generous vs. strict users ($a_m \in \mathbb{R}$) and popular vs. disliked movies ($b_n \in \mathbb{R}$) around the global mean $\mu = 3.5404$:

$$\hat{r}_{m,n} = \mu + a_m + b_n. \tag{1}$$

Assuming Gaussian rating noise $r_{m,n} \sim \mathcal{N}(\mu + a_m + b_n, \lambda^{-1})$ and Gaussian priors $a_m, b_n \sim \mathcal{N}(0, \gamma^{-1})$ gives the regularized squared-error loss function:

$$\mathcal{L}(a, b) = \frac{\lambda}{2} \sum_{(m,n)\in\Omega^{\text{train}}} (r_{m,n} - \mu - a_m - b_n)^2 + \frac{\gamma}{2} \sum_{m=1}^{M} a_m^2 + \frac{\gamma}{2} \sum_{n=1}^{N} b_n^2. \tag{2}$$

Alternating between fixing b to solve $\partial\mathcal{L}/\partial a_m = 0$ and fixing a to solve $\partial\mathcal{L}/\partial b_n = 0$ gives the exact updates:

$$a_m \leftarrow \frac{\lambda \sum_{n\in\Omega^{\text{train}}_m} (r_{m,n} - \mu - b_n)}{\lambda|\Omega^{\text{train}}_m| + \gamma}, \tag{3}$$

$$b_n \leftarrow \frac{\lambda \sum_{m\in\Omega^{n}_{\text{train}}} (r_{m,n} - \mu - a_m)}{\lambda|\Omega^{n}_{\text{train}}| + \gamma}. \tag{4}$$

Writing $\bar{e}_n = \frac{1}{|\Omega^n_{\text{train}}|} \sum_{m\in\Omega^n_{\text{train}}} (r_{m,n} - \mu - a_m)$ for the average residual on movie n gives $b_n = \frac{|\Omega^n_{\text{train}}|}{|\Omega^n_{\text{train}}|+\gamma/\lambda} \bar{e}_n$: when $|\Omega^n_{\text{train}}| \gg \gamma/\lambda$, $b_n \approx \bar{e}_n$, whereas when a movie has only 1 or 2 ratings, γ/λ pulls b_n toward 0. Also, summing the squared errors over users ($\mathcal{S}_{\text{user}}$) must give the exact same loss $\mathcal{L}_{\mathcal{U}}(a, b) = \mathcal{L}_{\mathcal{I}}(a, b)$ as summing over movies ($\mathcal{S}_{\text{movie}}$), which gives a simple way to check that both CSR matrices match.

3.2. Matrix Factorization with User and Movie Trait Vectors

To capture personal taste beyond simple user and movie averages, we add a K-dimensional trait vector $u_m \in \mathbb{R}^K$ for each user and $v_n \in \mathbb{R}^K$ for each movie (Mnih & Salakhutdinov, 2007; Koren et al., 2009):

$$\hat{r}_{m,n} = \mu + a_m + b_n + u_m^\top v_n, \quad u_m, v_n \in \mathbb{R}^K. \tag{5}$$

Placing independent Gaussian priors $u_m, v_n \sim \mathcal{N}(\mathbf{0}, \tau^{-1} I_K)$ with regularization $\tau > 0$ gives the full regularized loss over $\Theta = \{a, b, U, V\}$:

$$\mathcal{L}(\Theta) = \frac{\lambda}{2} \sum_{(m,n)\in\Omega^{\text{train}}} \left(r_{m,n} - \mu - a_m - b_n - u_m^\top v_n\right)^2 + \frac{\gamma}{2}\left(\|a\|_2^2 + \|b\|_2^2\right) + \frac{\tau}{2}\left(\sum_{m=1}^{M} \|u_m\|_2^2 + \sum_{n=1}^{N} \|v_n\|_2^2\right). \tag{6}$$

Although $\mathcal{L}(\Theta)$ is not convex when U and V vary together, holding (b, V) fixed turns $\mathcal{L}(\Theta)$ into a simple quadratic function of (a_m, u_m) for each user m separately. Setting the derivatives with respect to a_m and u_m to zero (Section A) gives the user updates:

$$a_m \leftarrow \frac{\lambda \sum_{n\in\Omega^{\text{train}}_m} \left(r_{m,n} - \mu - b_n - u_m^\top v_n\right)}{\lambda|\Omega^{\text{train}}_m| + \gamma}, \tag{7}$$

$$u_m \leftarrow \left(\lambda \sum_{n\in\Omega^{\text{train}}_m} v_n v_n^\top + \tau I_K\right)^{-1} \left(\lambda \sum_{n\in\Omega^{\text{train}}_m} \tilde{r}^{(u)}_{m,n} v_n\right), \tag{8}$$

where $\tilde{r}^{(u)}_{m,n} = r_{m,n} - \mu - a_m - b_n$. In the second half-step, holding (a, U) fixed on $\mathcal{S}_{\text{movie}}$ gives the movie updates:

$$b_n \leftarrow \frac{\lambda \sum_{m\in\Omega^{n}_{\text{train}}} \left(r_{m,n} - \mu - a_m - u_m^\top v_n\right)}{\lambda|\Omega^{n}_{\text{train}}| + \gamma}, \tag{9}$$

$$v_n \leftarrow \left(\lambda \sum_{m\in\Omega^{n}_{\text{train}}} u_m u_m^\top + \tau I_K\right)^{-1} \left(\lambda \sum_{m\in\Omega^{n}_{\text{train}}} \tilde{r}^{(v)}_{m,n} u_m\right), \tag{10}$$

where $\tilde{r}^{(v)}_{m,n} = r_{m,n} - \mu - a_m - b_n$. Because each step finds the exact minimum for the variables being updated, the loss $\mathcal{L}(\Theta)$ goes down at every step.

3.3. Adding Movie Genres: Altered Loss Function and ALS Steps

A limitation of standard matrix factorization appears when a movie is new ($|\Omega^n_{\text{train}}| = 0$) or has very few ratings: because the prior on v_n is centered at zero, Equation (10) sets $v_n = \mathbf{0}$ for any unrated movie, so $u_m^\top v_n = 0$ and the model cannot

personalize predictions. To address this, we include movie genres in the movie prior (Agarwal & Chen, 2009).

Each movie n is tagged with a set of genres $\mathcal{G}_n \subseteq \{1, \ldots, G\}$ ($G = 20$ genres total). Assigning each genre g a latent vector $f_g \in \mathbb{R}^K$, we center the Gaussian prior of v_n at the normalized sum of its genre vectors:

$$\bar{f}_n = \frac{1}{\sqrt{|\mathcal{G}_n|}} \sum_{g \in \mathcal{G}_n} f_g, \qquad v_n \,|\, F \;\sim\; \mathcal{N}\big(\bar{f}_n,\, \tau^{-1} I_K\big)\,, \tag{11}$$

where dividing by $\sqrt{|\mathcal{G}_n|}$ keeps the prior variance constant regardless of how many genre tags a movie has. Placing a prior $f_g \sim \mathcal{N}(\mathbf{0}, \tau_f^{-1} I_K)$ on the genre vectors gives our altered loss function:

$$\begin{aligned}\mathcal{L}_{\text{genre}} = \frac{\lambda}{2} \sum_{(m,n)\in\Omega^{\text{train}}} \big(r_{m,n} - \mu - a_m - b_n - u_m^\top v_n\big)^2 \\ + \frac{\gamma}{2}\left(\sum_{m=1}^{M} a_m^2 + \sum_{n=1}^{N} b_n^2\right) + \frac{\tau}{2}\sum_{m=1}^{M} \|u_m\|_2^2 \\ + \frac{\tau}{2}\sum_{n=1}^{N} \big\|v_n - \bar{f}_n\big\|_2^2 + \frac{\tau_f}{2}\sum_{g=1}^{G} \|f_g\|_2^2. \end{aligned} \tag{12}$$

Optimization Steps. Because $\frac{\tau}{2}\sum_{n=1}^{N} \|v_n - \bar{f}_n\|_2^2$ is quadratic in both v_n and f_g, each ALS epoch splits into three alternating steps (Section A):

1. Step 1: Update Users (a_m, u_m): Identical to Equations (7) and (8).

2. Step 2: Update Movies (b_n, v_n): Bias b_n follows Equation (9), while setting $\nabla_{v_n} \mathcal{L}_{\text{genre}} = \mathbf{0}$ adds $\tau \bar{f}_n$ to the right-hand side of the movie vector update (writing $\Omega^n = \Omega^n_{\text{train}}$):

$$v_n \leftarrow \left(\lambda \sum_{m\in\Omega^n} u_m u_m^\top + \tau I_K\right)^{-1} \left(\lambda \sum_{m\in\Omega^n} \tilde{r}^{(v)}_{m,n} u_m + \tau \bar{f}_n\right). \tag{13}$$

For a *new movie* ($|\Omega^n| = 0$), the rating sum is zero and $v_n \leftarrow (\tau I_K)^{-1}(\tau \bar{f}_n) = \bar{f}_n$, so the movie immediately gets a trait vector from its genres. For a *popular movie* ($|\Omega^n| \gg \tau/\lambda$), the rating sum is much larger than τI_K and v_n is learned mostly from user ratings.

3. Step 3: Update Genre Vectors (f_g): Holding $V = [v_1, \ldots, v_N]^\top \in \mathbb{R}^{N\times K}$ fixed, let $B \in \mathbb{R}^{N\times G}$ have entries $B_{n,g} = |\mathcal{G}_n|^{-1/2}$ if $g \in \mathcal{G}_n$ and 0 otherwise, so $\bar{f}_n = (BF)^\top_{n,:}$. Minimizing Equation (12) over $F = [f_1, \ldots, f_G]^\top \in \mathbb{R}^{G\times K}$ gives the ridge regression update:

$$F \leftarrow \left(\tau B^\top B + \tau_f I_G\right)^{-1} \left(\tau B^\top V\right), \tag{14}$$

or one genre g at a time as derived in Section A. Because $G = 20$, solving this 20×20 system takes less than 2 ms per epoch.

3.4. Making ALS Fast on 32 Million Ratings

Running Equations (8), (10) and (13) on 25.6M training ratings per epoch requires building and solving $M + N = 285{,}380$ systems of size $K \times K$ in every iteration:

- **Why Batched Array Operations Are Slow:** Computing outer products $v_n v_n^\top \in \mathbb{R}^{K\times K}$ across batches of rows allocates a 3D array of shape $|\Omega_{\text{batch}}| \times K \times K$ ($[0.3, 3.6]$ GB of temporary RAM per batch), which slows down training due to large memory allocations and cache misses (Section B).

- **Parallel CSR Loops with In-Place Cholesky:** Instead, because user and movie updates have the same mathematical form, we run a single parallel loop over the rows of the active CSR matrix ($\mathcal{S}_{\text{user}}$ or $\mathcal{S}_{\text{movie}}$) across CPU threads. Each thread updates one user or movie at a time using a small thread-local matrix $A \in \mathbb{R}^{K\times K}$ and vector $b \in \mathbb{R}^K$ in L1 cache (2 KB for $K = 16$), adding $\lambda v_n v_n^\top$ into the lower triangle of A. Because $A = \lambda \sum v_n v_n^\top + \tau I_K$ is symmetric positive-definite for $\tau > 0$, we factor $A = LL^\top$ in-place via Cholesky decomposition ($L_{i,i} = \sqrt{A_{i,i} - \sum_{p<i} L_{i,p}^2}$ and $L_{i,j} = L_{j,j}^{-1}(A_{i,j} - \sum_{p<j} L_{i,p}L_{j,p})$ for $i > j$) followed by forward and backward substitution, using half as many operations as Gaussian elimination without temporary arrays.

4. Experimental Results

4.1. Train/Test Split and Evaluation Setup

We split Ω randomly using an $80\%/20\%$ coin flip (seed 42) into Ω^{train} (25,604,307 ratings on 32M; 4,000,446 on 5M) and Ω^{test} (6,395,897 on 32M; 999,554 on 5M), stored in dual CSR format. To test how the models handle new movies, we also create a cold-start split where all training ratings are held out ($|\Omega^n_{\text{train}}| = 0$) for 500 movies (73,217 test ratings). Models are evaluated by Root Mean Squared Error, $\text{RMSE}(\Omega') = \sqrt{|\Omega'|^{-1} \sum_{(m,n)\in\Omega'} (r_{m,n} - \hat{r}_{m,n})^2}$, with latent vectors initialized from $\mathcal{N}(0, (0.1/\sqrt{K})^2)$.

4.2. Bias-Only ALS and Matrix Factorization Convergence

Figure 2 shows the convergence of the bias-only model ($\lambda = 1.0, \gamma = 0.05$) over 25 ALS iterations. On the 32M dataset, one iteration takes 62.3 ms (11.6 ms on 5M). In Figure 2(a), the loss computed over $\mathcal{S}_{\text{user}}$ (lines) and over $\mathcal{S}_{\text{movie}}$ (markers) match at every step up to a relative difference of 5.28×10^{-15} (float64 rounding), showing that both CSR views store the same split. In Figure 2(b), the curves flatten after 5 iterations: on MovieLens-32M, train RMSE drops

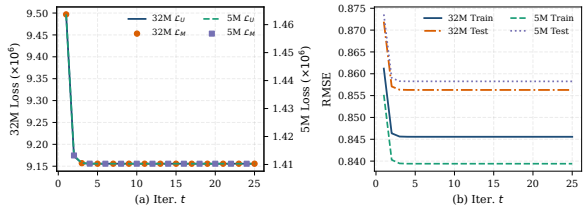


Figure 2. Training curves for the bias-only ALS model ($\lambda = 1.0, \gamma = 0.05$) over 25 iterations on MovieLens-32M and MovieLens-5M. **(a)** Regularized loss $\mathcal{L}(a, b)$ computed on $\mathcal{S}_{\text{user}}$ (lines) and $\mathcal{S}_{\text{movie}}$ (markers), showing exact agreement. **(b)** Train and test RMSE.

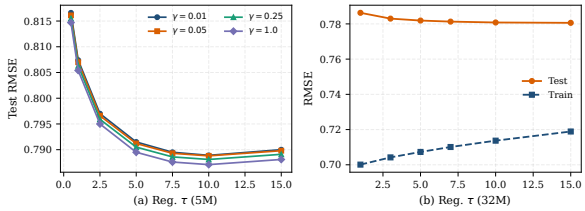


Figure 4. Hyperparameter search for $K = 10$ ($\lambda = 1.0$). **(a)** Grid search on MovieLens-5M over $\tau \in [0.5, 15.0]$ and $\gamma \in \{0.01, 0.05, 0.25, 1.0\}$. **(b)** Train and test RMSE on MovieLens-32M across τ, showing overfitting when τ is too small.

from 0.8612 to 0.8456 and test RMSE drops from 0.8719 to 0.8563 (0.8394 and 0.8583 on 5M), which is 19.1% lower than predicting the global mean μ (1.0590).

Figure 3 shows the training progress when we add $K = 10$ latent dimensions for 20 iterations. As expected from the ALS update rules, the regularized loss in Figure 3(a) goes down at every step (from 8.28×10^6 to 6.83×10^6 on 32M), while test RMSE in Figure 3(b) drops to 0.7805 (train RMSE 0.7082), improving over the bias-only model by 0.0758 stars. Most of the drop happens in the first 12 iterations, and between iterations 15 (0.7810) and 20 (0.7805) test RMSE changes by 0.0005 stars, so 15 to 20 ALS iterations are enough.

4.3. Choosing Hyperparameters (λ, γ, τ), Rank K, and Overfitting

Because multiplying (λ, γ, τ) by a constant does not change the minimum of Equation (6), we fix $\lambda = 1.0$ and tune γ and τ (Figure 4). Two results stand out:

1. **Vector Regularization τ Matters More Than γ:** Changing the bias penalty γ from 0.01 to 1.0 changes test RMSE by less than 0.002 stars because each user and movie has only one bias parameter. In contrast, U and V contain $K(M+N)$ parameters (2.85M numbers at $K = 10$ on 32M). When τ is too small ($\tau = 0.5$ on 5M), the model fits the training ratings closely (train RMSE 0.6779) but overfits on the test set (test RMSE 0.8161). The lowest test RMSE is reached at $\tau = 5.0$ on 5M and $\tau = 7.0$ on 32M, with $\gamma \in \{0.05, 0.1\}$.

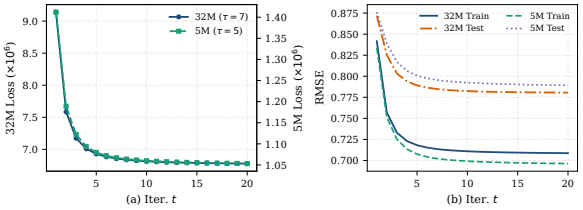


Figure 3. Training curves for matrix factorization with $K = 10$ ($\lambda = 1.0, \gamma = 0.05$) over 20 iterations on MovieLens-32M ($\tau = 7.0$) and MovieLens-5M ($\tau = 5.0$). **(a)** Regularized loss $\mathcal{L}(\Theta)$ decreasing monotonically. **(b)** Train and test RMSE over iterations.

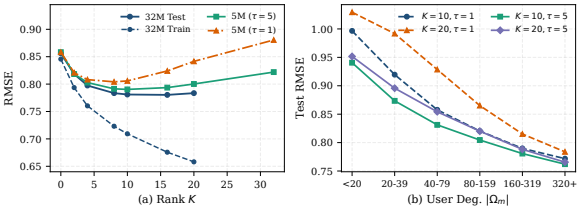


Figure 5. Effect of latent dimension K and user activity on overfitting. **(a)** Train/test RMSE across $K \in [0, 32]$ for tuned vs. weak τ. **(b)** Test RMSE broken down by user training ratings $|\Omega_m^{\text{train}}|$, showing how $K = 20$ hurts low-activity users when τ is small.

2. **Smaller Gains as K Gets Larger:** Figure 5(a) and Table 2 compare 15-epoch models across $K \in \{0, 2, 4, 8, 10, 16, 20, 32\}$. Going from $K = 0$ (0.8563) to $K = 2$ (0.8183), $K = 4$ (0.7974), and $K = 8$ (0.7835) reduces test error on 32M. Past $K = 10$ (0.7810), adding more dimensions helps much less: $K = 16$ reaches a test RMSE of 0.7803 on 32M ($K = 10$ is lowest on 5M at 0.7903), and by $K = 20$ test RMSE increases again (0.7835).

To see why $K = 20$ starts to overfit compared to $K = 10$, Figure 5(b) breaks down test RMSE by user training ratings $|\Omega_m^{\text{train}}|$. Under weak regularization ($\tau = 1.0$), increasing K from 10 to 20 causes test RMSE for users with $|\Omega_m^{\text{train}}| \in [20, 39]$ to increase by $+0.0722$ stars ($0.9196 \to 0.9918$), compared to $+0.0120$ stars ($0.7717 \to 0.7837$) for active users with $|\Omega_m^{\text{train}}| \geq 320$: fitting $K = 20$ parameters to a user with 20 ratings gives the model as many parameters as ratings, so it fits noise unless τ keeps u_m small. Raising τ from 1.0 to 5.0 removes most of this overfitting gap across all user groups (Figure 5b).

4.4. Runtime Speedup on 32M Ratings

Figure 6 and Table 2 compare our parallel CSR Cholesky solver against single-thread CSR and batched array solvers. On MovieLens-5M at $K = 10$, the batched solver takes 12.73 s/epoch, the single-thread CSR solver takes 0.337 s, and the 12-thread CSR solver takes 0.183 s/epoch (69.6×

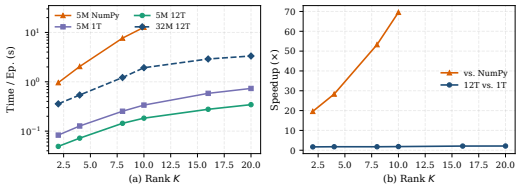


Figure 6. Runtime comparison across $K \in \{2, 4, 8, 10, 16, 20\}$. **(a)** Time per ALS epoch (seconds, log scale) comparing batched array operations against 1-thread and 12-thread CSR Cholesky solvers. **(b)** Speedup factor of the 12-thread CSR Cholesky solver.

Table 2. Train/Test RMSE (15 epochs) and runtime per epoch across latent dimensions K on MovieLens-5M ($\tau = 5.0$) and MovieLens-32M ($\tau = 7.0, \lambda = 1.0, \gamma = 0.05$).

Rank	MovieLens-5M		MovieLens-32M		
K	Test	Time/Ep	Train	Test	Time/Ep
0 (Bias)	0.8583	0.012 s	0.8456	0.8563	0.062 s
2	0.8183	0.049 s	0.7936	0.8183	0.357 s
4	0.8028	0.072 s	0.7603	0.7974	0.539 s
8	0.7914	0.144 s	0.7231	0.7835	1.219 s
10	0.7903	0.183 s	0.7093	0.7810	1.927 s
16	0.7936	0.277 s	0.6757	**0.7803**	2.899 s
20	0.8002	0.344 s	0.6581	0.7835	3.329 s

faster than batched array operations). On MovieLens-32M, one epoch takes 0.357 s at $K = 2$, 1.927 s at $K = 10$, and 2.899 s at $K = 16$, so a $K = 16$ model trains on all 32 million ratings in under a minute on a laptop.

4.5. Visualizing 2D Movie Trait Vectors

We train our matrix factorization model with $K = 2$ latent dimensions ($\lambda = 1.0, \gamma = 0.1, \tau = 5.0$) for 18 epochs on MovieLens-32M, reaching train RMSE 0.7931 and test RMSE 0.8180. Figure 7 plots the learned 2D vectors $v_n = (v_{n,1}, v_{n,2})$ for a selection of movies. Even though the model only uses user IDs, movie IDs, and ratings, the 2D space groups related movies together:

1. **Sequels and Similar Movies Group Together:** Both *Lord of the Rings* movies land close together in the upper region: *The Fellowship of the Ring* at $(+0.35, +1.27)$ and *The Return of the King* at $(+0.25, +1.37)$, below *Inception* $(+0.39, +2.22)$. Similarly, Disney/Pixar animated films (*Toy Story* at $(+0.09, -0.66)$ and *The Lion King* at $(-0.58, -0.19)$) sit near each other in the family animation region.

2. **Opposite Genres Sit in Opposite Parts of the Space:** Moving from left to right along $v_{n,1}$ goes from romance dramas on the left (*Twilight* $(-2.27, +1.25)$, *Titanic* $(-1.16, +0.94)$) to crime dramas on the right (*The Godfather* $(+1.72, +0.43)$, *Pulp Fiction* $(+2.14, +0.39)$, *Fargo* $(+2.16, +0.24)$). Moving vertically along $v_{n,2}$ goes from sci-fi and fantasy at the top ($v_{n,2} > +1.2$) down to slapstick comedies at the bottom (*Dumb & Dumber* $(-0.47, -3.66)$).

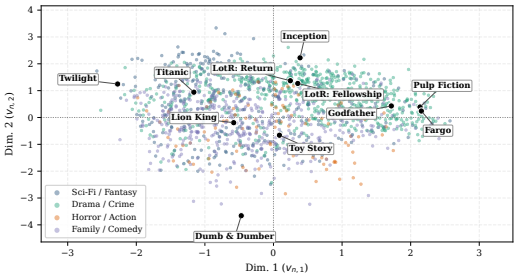


Figure 7. 2D movie trait vectors $v_n \in \mathbb{R}^2$ learned from ratings on MovieLens-32M ($K = 2$, 18 epochs, test RMSE $= 0.8180$). Similar movies group together while opposite genres sit on opposite sides.

4.6. Dummy User Recommendations (*Lord of the Rings*)

Suppose a new user gives a single 5.0-star rating to *The Lord of the Rings: The Fellowship of the Ring (2001)* ($n^* = 339$, $|\Omega^{n^*}_{\text{train}}| = 58{,}491$, $b_{n^*} = +0.5191$). Keeping all movie parameters (b, V) from our $K = 16$ MovieLens-32M model fixed and running the user update Equations (7) and (8) on this single rating points u_{new} along v_{n^*}:

$$u_{\text{new}} = \frac{\lambda(r_{\text{new},n^*} - \mu - a_{\text{new}} - b_{n^*})}{\lambda \|v_{n^*}\|_2^2 + \tau} v_{n^*}, \qquad (15)$$

giving $a_{\text{new}} = +0.7024$ and $\|u_{\text{new}}\|_2 = 0.0281$. Because $\tau = 5.0$ keeps $\|u_{\text{new}}\|_2$ small for a user with only one rating, ranking movies via $s_\alpha(\text{new}, n) = u_{\text{new}}^\top v_n + \alpha b_n$ depends on the bias weight α (Table 3; full Top-10 across Settings A, B, and C in Table 7):

- **Why Full Bias ($\alpha = 1.0$) Fails:** The largest dot product $u_{\text{new}}^\top v_n$ across all movies is $+0.2170$, whereas movies with $|\Omega^n_{\text{train}}| \in \{1, 2\}$ that got 5.0 stars from a strict rater ($a_m \approx -2.1$) end up with $b_n \approx +3.27$ and appear at the top of the list even though their dot product with the user is near zero (0.0002).

- **Downweighting the Bias ($\alpha = 0.05$) Recovers the Trilogy:** Lowering α to 0.05 puts *Return of the King* and *The Two Towers* at ranks 1 and 2 across all movies (Setting B in Table 7), and requiring $|\Omega^n_{\text{train}}| \geq 100$ (Setting C in Table 3) places the five other Middle-earth movies (*Lord of the Rings* and *The Hobbit*) at ranks 1 to 5, followed by *Braveheart* and four *Harry Potter* movies. A dummy user rating *Toy Story (1995)* 5.0 stars similarly gets *Toy Story 3*, *Toy Story 2*, *Finding Nemo*, *The Incredibles*, and *WALL·E* at ranks 1 to 5.

Table 3. Top recommendations for a dummy user rating *LOTR: Fellowship of the Ring* 5.0 stars ($K = 16$, ML-32M) under $s_\alpha(\text{new}, n) = u_{\text{new}}^\top v_n + \alpha b_n$ (full Top-10 across all 3 settings in Table 7).

Rk	Recommended Title	$u_{\text{new}}^\top v_n$	b_n	$\lvert\Omega^n_{\text{train}}\rvert$
Setting A: Full Bias $\alpha = 1.0$ (All Movies)				
1	Hello & Goodbye (2018)	0.0002	+3.271	1
2	SpongeBob: Heroes of Bikini Bottom	−0.0067	+3.256	2
3	Wesley (2009)	0.0002	+3.187	1
Setting C: Downweighted Bias $\alpha = 0.05$ ($\lvert\Omega^n_{train}\rvert \geq 100$)				
1	**LOTR: The Return of the King (2003)**	0.2170	+0.510	54,037
2	**LOTR: The Two Towers (2002)**	0.2143	+0.478	54,016
3	**Hobbit: An Unexpected Journey (2012)**	0.1590	−0.026	12,821
4	**Hobbit: Desolation of Smaug (2013)**	0.1550	−0.114	9,701
5	**Hobbit: Battle of Five Armies (2014)**	0.1523	−0.237	6,951
6	Braveheart (1995)	0.1114	+0.262	55,834
7	Harry Potter & Deathly Hallows Pt. 2	0.1110	+0.115	16,242

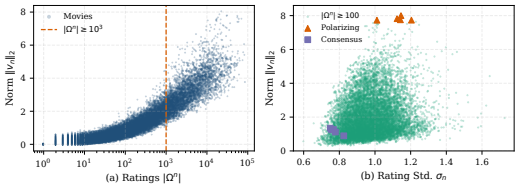


Figure 8. Movie vector length $\|v_n\|_2$ ($K = 16$) and polarization on MovieLens-32M. **(a)** Vector length $\|v_n\|_2$ vs. training count $|\Omega^n_{\text{train}}|$. **(b)** Top polarizing vs. least polarizing movies among movies with $|\Omega^n_{\text{train}}| \geq 1{,}000$.

Table 4. Top-5 most polarizing (largest $\|v_n\|_2$) and least polarizing (smallest $\|v_n\|_2$) movies with $|\Omega^n_{\text{train}}| \geq 1{,}000$ ($K = 16$, ML-32M; full Top-10 in Table 8).

Rk	Movie Title	$\|v_n\|_2$	b_n	$\lvert\Omega^n_{\text{train}}\rvert$	$\sigma(r)$
Most Polarizing Movies ($\max \|v_n\|_2$, $\lvert\Omega^n_{train}\rvert \geq 1{,}000$)					
1	Star Wars Ep. II: Attack of the Clones	8.000	−0.577	21,907	1.146
2	Star Wars Ep. III: Revenge of the Sith	7.819	−0.267	19,751	1.124
3	Star Wars Ep. I: The Phantom Menace	7.754	−0.564	30,338	1.143
4	Dumb & Dumber (1994)	7.732	−0.629	29,483	1.204
5	LOTR: The Return of the King (2003)	7.730	+0.510	54,037	1.011
Least Polarizing Movies ($\min \|v_n\|_2$, $\lvert\Omega^n_{train}\rvert \geq 1{,}000$)					
1	Cobb (1994)	0.892	−0.266	1,023	0.825
2	Pirates of Silicon Valley (1999)	1.119	+0.144	1,061	0.778
3	Sea of Love (1989)	1.248	+0.038	1,038	0.757
4	Black Book (Zwartboek) (2006)	1.263	+0.420	1,014	0.767
5	Run Silent Run Deep (1958)	1.347	+0.211	1,089	0.749

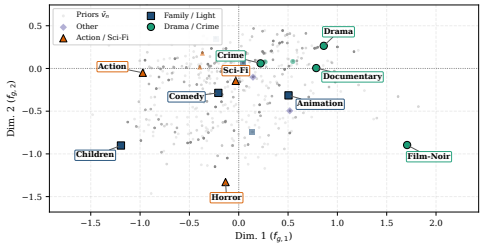


Figure 9. 2D genre trait vectors $f_g \in \mathbb{R}^2$ learned on MovieLens-32M ($K = 2$, 18 epochs, test RMSE $= 0.8175$). Circle size reflects the number of movies in each genre.

4.7. Most and Least Polarizing Movies

In our model, b_n captures whether a movie is liked on average by everyone, while $v_n \in \mathbb{R}^K$ captures how much a movie's rating depends on a user's taste u_m. When user vectors have covariance $\text{Cov}(u_m) \approx \sigma_u^2 I_K$, the variance of the personalized score across users is proportional to the squared vector length, $\text{Var}_m(u_m^\top v_n) \approx \sigma_u^2 \|v_n\|_2^2$. Thus movies with a large $\|v_n\|_2$ are polarizing (liked by viewers with $u_m^\top v_n \gg 0$ and disliked by viewers with $u_m^\top v_n \ll 0$), whereas movies with a small $\|v_n\|_2$ are least polarizing (rated similarly across viewers).

Filtering Out Small Sample Sizes: As shown in Figure 8(a), a movie with $|\Omega^n_{\text{train}}| \in \{1, 2\}$ gets pulled to $\|v_n\|_2 \approx 0$ by regularization τ, while a movie with $|\Omega^n_{\text{train}}| \in [20, 30]$ can get a large $\|v_n\|_2$ by fitting a few extreme ratings. To avoid small-sample noise, we only compare movies with at least $|\Omega^n_{\text{train}}| \geq 1{,}000$ training ratings (4,394 movies in MovieLens-32M). Table 4 and Figure 8(b) (full Top-10 in Table 8) show:

- **Most Polarizing Movies ($\|v_n\|_2 \in [7.54, 8.00]$):** Movies that divide viewers ($\sigma(r) \in [1.12, 1.23]$), such as the *Star Wars* prequels (*Attack of the Clones*, *Revenge of the Sith*, *The Phantom Menace*), the comedy *Dumb & Dumber*, and the horror film *The Blair Witch Project*, along with fantasy and sci-fi series (*Lord of the Rings*, *Star Wars*, *Harry Potter*) that get high scores from fans of the genre and lower scores from other viewers.

- **Least Polarizing Movies ($\|v_n\|_2 \in [0.89, 1.40]$):** Biographies, historical dramas, and thrillers (*Cobb*, *Pirates of Silicon Valley*, *Sea of Love*, *Black Book*, *Run Silent Run Deep*) with smaller rating standard deviations ($\sigma(r) \in [0.70, 0.83]$) that receive similar scores across different groups of viewers.

4.8. 2D Genre Embeddings and Cold-Start Results

We first train the genre-informed model with $K = 2$ on MovieLens-32M ($\text{RMSE}_{\text{train}} = 0.7927$, $\text{RMSE}_{\text{test}} = 0.8175$) and plot the 20 learned genre vectors $f_g \in \mathbb{R}^2$ in Figure 9. The genre positions line up with the movie map in Figure 7: *Action* $(-0.97, -0.05)$, *Adventure* $(-0.39, +0.02)$, and *Sci-Fi* sit on the left side; *Drama* $(+0.86, +0.27)$, *Documentary* $(+0.79, 0.00)$, *Romance*, and *Film-Noir* $(+1.71, -0.90)$ sit on the right side; and *Children* $(-1.19, -0.90)$ and *Animation* sit together in the lower-left family region while *Horror* $(-0.13, -1.33)$ sits at the bottom.

Finally, we compare standard ALS against Genre-ALS at $K = 16$ ($\lambda = 1.0, \gamma = 0.1, \tau = 5.0, \tau_f = 1.0$) on the full MovieLens-32M split and on the cold-start split (500

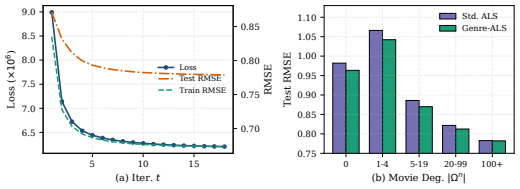


Figure 10. Evaluation of the Genre-ALS model ($K = 16$). **(a)** Loss $\mathcal{L}_{\text{genre}}$ and test RMSE on MovieLens-32M (0.7790). **(b)** Test RMSE grouped by movie training ratings $|\Omega^n_{\text{train}}|$.

Table 5. Test RMSE comparison between Standard ALS and Genre-ALS at $K = 16$, grouped by the number of training ratings per movie $|\Omega^n_{\text{train}}|$.

Movie Group $\lvert\Omega^n_{\text{train}}\rvert$	**Test** $\lvert\Omega\rvert$	**Std. ALS**	**Genre-ALS**	**ΔRMSE**
Controlled Cold-Start Split ($K = 16$)				
0 (Cold-Start)	73,217	0.9819	**0.9634**	**−0.0186**
[1, 4] (Very Rare)	9,304	1.0661	**1.0422**	**−0.0238**
[5, 19] (Rare)	18,940	0.8859	**0.8698**	**−0.0161**
[20, 99] (Moderate)	61,270	0.8217	**0.8125**	**−0.0092**
≥ 100 (Popular)	836,823	0.7827	**0.7820**	**−0.0007**
Full MovieLens-32M Split ($K = 16$)				
[1, 4] (Very Rare)	18,533	1.0237	**1.0026**	**−0.0212**
[5, 19] (Rare)	42,171	0.8829	**0.8622**	**−0.0207**
[20, 99] (Moderate)	111,407	0.8340	**0.8194**	**−0.0146**
≥ 100 (Popular)	6,219,001	0.7767	**0.7767**	−0.0000
All Test (32M)	**6,395,897**	0.7796	**0.7790**	**−0.0005**

held-out movies with $|\Omega^n_{\text{train}}| = 0$). Figure 10 and Table 5 show:

1. **Lower Error on Cold-Start and Rare Movies:** When a movie has 0 training ratings, standard ALS sets $v_n = \mathbf{0}$ (RMSE = 0.9819), whereas Genre-ALS uses the genre average $v_n = \bar{f}_n$ to match the movie's genres with each user's vector u_m, lowering cold-start test RMSE by -0.0186 stars (0.9634). On very rare ($|\Omega^n_{\text{train}}| \in [1, 4]$) and rare ($|\Omega^n_{\text{train}}| \in [5, 19]$) movies in MovieLens-32M, Genre-ALS lowers test RMSE by -0.0212 and -0.0207 stars (-0.0238 and -0.0161 on the cold-start split).

2. **Same Accuracy on Popular Movies:** For popular movies ($|\Omega^n_{\text{train}}| \geq 100$, 97.2% of test ratings), user ratings outweigh the genre prior and both models reach 0.7767. Overall 32M test RMSE changes from 0.7796 to 0.7790, with most of the improvement coming from rare movies.

5. Conclusion

In this report, we built and tested a matrix factorization recommender system on the MovieLens-32M dataset (32,000,204 ratings). By storing the sparse rating table in two CSR matrices ($\mathcal{S}_{\text{user}}$ and $\mathcal{S}_{\text{movie}}$) and running the in-place Cholesky updates in parallel across CPU threads, we reduced training time by 69.6× compared to batched array solvers and trained a 16-dimensional model on 32 million ratings in under 3 seconds per epoch. Our experiments showed that adding latent vectors lowers test RMSE from 0.8563 (biases only) to 0.7803 ($K = 16$), that regularization τ is needed to keep higher-rank models ($K = 20$) from overfitting users with few ratings, and that 2D movie and genre vectors group related films together. Finally, down-weighting movie biases gives sensible recommendations for single-rating users, and adding movie genres to the movie prior helps predictions on cold-start and rarely rated movies.

References

Agarwal, D. and Chen, B.-C. Regression-based latent factor models. In *Proceedings of the 15th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pp. 19–28. ACM, 2009.

Clauset, A., Shalizi, C. R., and Newman, M. E. J. Power-law distributions in empirical data. *SIAM Review*, 51(4): 661–703, 2009.

Harper, F. M. and Konstan, J. A. The MovieLens datasets: History and context. *ACM Transactions on Interactive Intelligent Systems (TiiS)*, 5(4):1–19, 2015.

Koren, Y., Bell, R., and Volinsky, C. Matrix factorization techniques for recommender systems. *Computer*, 42(8): 30–37, 2009.

Mnih, A. and Salakhutdinov, R. R. Probabilistic matrix factorization. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 20, pp. 1257–1264, 2007.

Zhou, Y., Wilkinson, D., Schreiber, R., and Pan, R. Large-scale parallel collaborative filtering for the Netflix prize. In *International Conference on Algorithmic Applications in Management (AAIM)*, pp. 337–348. Springer, 2008.

A. Detailed Mathematical Derivations of ALS Updates

In this appendix, we give the complete step-by-step derivations of the ALS coordinate updates for the bias-only model (Section 3.1), the matrix factorization model (Section 3.2), and the genre-informed model (Section 3.3). Throughout this section, we write $\Omega_m = \Omega_m^{\text{train}}$ and $\Omega^n = \Omega_{\text{train}}^n$ for the training ratings of user m and movie n.

A.1. Derivations for User and Movie Biases and Trait Vectors

Consider the general genre-informed loss function $\mathcal{L}_{\text{genre}}(\Theta, F)$ in Equation (12). Setting $f_g = \mathbf{0}$ recovers the standard matrix factorization loss $\mathcal{L}(\Theta)$ in Equation (6), and setting $K = 0$ recovers the bias-only loss $\mathcal{L}(a, b)$ in Equation (2).

Proposition A.1 (User Bias and User Trait Vector Updates). *For any user $m \in \{1, \dots, M\}$ with training ratings Ω_m, holding all movie parameters $\{b_n, v_n\}_{n=1}^N$ fixed, the partial derivatives of $\mathcal{L}_{genre}$ with respect to $a_m \in \mathbb{R}$ and $u_m \in \mathbb{R}^K$ are:*

$$\frac{\partial \mathcal{L}_{\text{genre}}}{\partial a_m} = -\lambda \sum_{n \in \Omega_m} \left(r_{m,n} - \mu - a_m - b_n - u_m^\top v_n\right) + \gamma a_m, \tag{16}$$

$$\nabla_{u_m} \mathcal{L}_{\text{genre}} = -\lambda \sum_{n \in \Omega_m} \left(r_{m,n} - \mu - a_m - b_n - u_m^\top v_n\right) v_n + \tau u_m. \tag{17}$$

Setting $\partial \mathcal{L}_{genre}/\partial a_m = 0$ and $\nabla_{u_m} \mathcal{L}_{genre} = \mathbf{0}$ gives the exact update rules in Equations (7) and (8). In addition, the Hessian matrix with respect to u_m is

$$\nabla^2_{u_m} \mathcal{L}_{\text{genre}} = \lambda \sum_{n \in \Omega_m} v_n v_n^\top + \tau I_K \succeq \tau I_K \succ 0, \tag{18}$$

which is strictly positive-definite whenever $\tau > 0$, ensuring a unique minimum.

Proof. Keeping only the terms in Equation (12) that depend on user m, let $e_{m,n}^{(-a_m)} = r_{m,n} - \mu - b_n - u_m^\top v_n$. Differentiating $\ell_m(a_m) = \frac{\lambda}{2} \sum_{n \in \Omega_m} (e_{m,n}^{(-a_m)} - a_m)^2 + \frac{\gamma}{2} a_m^2$ with respect to a_m and setting it to zero gives:

$$\frac{\partial \ell_m}{\partial a_m} = -\lambda \sum_{n \in \Omega_m} e_{m,n}^{(-a_m)} + (\lambda |\Omega_m| + \gamma)\, a_m = 0 \iff a_m^* = \frac{\lambda \sum_{n \in \Omega_m} \left(r_{m,n} - \mu - b_n - u_m^\top v_n\right)}{\lambda |\Omega_m| + \gamma}. \tag{19}$$

Next, for u_m, let $e_{m,n}^{(-u_m)} = r_{m,n} - \mu - a_m - b_n$. Expanding the quadratic terms in u_m gives:

$$\ell_m(u_m) = \frac{\lambda}{2} \sum_{n \in \Omega_m} \left(u_m^\top v_n v_n^\top u_m - 2 e_{m,n}^{(-u_m)} v_n^\top u_m + (e_{m,n}^{(-u_m)})^2 \right) + \frac{\tau}{2} u_m^\top I_K u_m. \tag{20}$$

Setting $\nabla_{u_m} \ell_m(u_m) = \mathbf{0}$ yields the $K \times K$ linear system:

$$\left(\lambda \sum_{n \in \Omega_m} v_n v_n^\top + \tau I_K \right) u_m^* = \lambda \sum_{n \in \Omega_m} e_{m,n}^{(-u_m)} v_n. \tag{21}$$

Since each outer product $v_n v_n^\top$ is positive semi-definite ($x^\top v_n v_n^\top x = (v_n^\top x)^2 \geq 0$) and $\tau > 0$, the matrix $A_m = \lambda \sum_{n \in \Omega_m} v_n v_n^\top + \tau I_K$ is symmetric positive-definite and can always be factored via Cholesky decomposition $A_m = L_m L_m^\top$. □

A.2. Derivations for Movie Trait Vectors with Genres and Genre Embeddings

Proposition A.2 (Movie Vector Update with Genre Prior). *Holding user parameters $\{a_m, u_m\}_{m=1}^M$, movie biases $\{b_n\}_{n=1}^N$, and genre vectors $F = [f_1, \dots, f_G]^\top \in \mathbb{R}^{G \times K}$ fixed, the unique minimizer of $\mathcal{L}_{genre}$ with respect to $v_n \in \mathbb{R}^K$ is given by Equation (13).*

Proof. Let $e_{m,n}^{(-v_n)} = r_{m,n} - \mu - a_m - b_n$ and $\bar{f}_n = |\mathcal{G}_n|^{-1/2} \sum_{g \in \mathcal{G}_n} f_g$. Keeping only the terms of $\mathcal{L}_{\text{genre}}$ that depend on v_n, $\ell^n(v_n) = \frac{\lambda}{2} \sum_{m \in \Omega^n} (e_{m,n}^{(-v_n)} - u_m^\top v_n)^2 + \frac{\tau}{2} \|v_n - \bar{f}_n\|_2^2$, and setting $\nabla_{v_n} \ell^n(v_n) = -\lambda \sum_{m \in \Omega^n} (e_{m,n}^{(-v_n)} - u_m^\top v_n) u_m +$

$\tau(v_n - \bar{f}_n) = \mathbf{0}$ gives:

$$\left(\lambda \sum_{m\in\Omega^n} u_m u_m^\top + \tau I_K\right) v_n^* = \lambda \sum_{m\in\Omega^n} e_{m,n}^{(-v_n)} u_m + \tau \bar{f}_n, \tag{22}$$

which gives Equation (13). □

Proposition A.3 (Genre Vector Update). *Holding the movie vectors $V = [v_1, \dots, v_N]^\top \in \mathbb{R}^{N\times K}$ fixed, the genre part of the loss function*

$$\mathcal{L}_F(F) = \frac{\tau}{2} \sum_{n=1}^{N} \left\| v_n - \frac{1}{\sqrt{|\mathcal{G}_n|}} \sum_{g\in\mathcal{G}_n} f_g \right\|_2^2 + \frac{\tau_f}{2} \sum_{g=1}^{G} \|f_g\|_2^2 \tag{23}$$

is minimized by $F^* = (\tau B^\top B + \tau_f I_G)^{-1}(\tau B^\top V)$ in Equation (14), or for a single genre g over $\mathcal{I}_g = \{n : g \in \mathcal{G}_n\}$ by:

$$f_g \leftarrow \frac{\tau \sum_{n\in\mathcal{I}_g} |\mathcal{G}_n|^{-1/2}\, \tilde{v}_n^{(-g)}}{\tau \sum_{n\in\mathcal{I}_g} |\mathcal{G}_n|^{-1} + \tau_f}, \qquad \tilde{v}_n^{(-g)} = v_n - \frac{1}{\sqrt{|\mathcal{G}_n|}} \sum_{g'\in\mathcal{G}_n\setminus\{g\}} f_{g'}. \tag{24}$$

Proof. Using the matrix $B \in \mathbb{R}^{N\times G}$ with $B_{n,g} = |\mathcal{G}_n|^{-1/2}$ if $g \in \mathcal{G}_n$ and 0 otherwise, row n of BF is $\bar{f}_n^\top$. Therefore we can write $\mathcal{L}_F(F)$ in matrix form as:

$$\mathcal{L}_F(F) = \frac{\tau}{2}\|V - BF\|_F^2 + \frac{\tau_f}{2}\|F\|_F^2 = \frac{1}{2}\operatorname{Tr}\big(\tau(V - BF)^\top(V - BF) + \tau_f F^\top F\big). \tag{25}$$

Setting the matrix derivative $\nabla_F \mathcal{L}_F(F) = -\tau B^\top(V - BF) + \tau_f F = \mathbf{0}$ gives $(\tau B^\top B + \tau_f I_G)F^* = \tau B^\top V$. If instead we update one genre g at a time with all other genres $g' \neq g$ fixed, we write $v_n - \bar{f}_n = \tilde{v}_n^{(-g)} - |\mathcal{G}_n|^{-1/2} f_g$ for each movie $n \in \mathcal{I}_g$, and setting $\nabla_{f_g}\mathcal{L}_F = -\tau \sum_{n\in\mathcal{I}_g} |\mathcal{G}_n|^{-1/2}(\tilde{v}_n^{(-g)} - |\mathcal{G}_n|^{-1/2} f_g) + \tau_f f_g = \mathbf{0}$ gives:

$$\left(\tau \sum_{n\in\mathcal{I}_g} \frac{1}{|\mathcal{G}_n|} + \tau_f\right) f_g^* = \tau \sum_{n\in\mathcal{I}_g} \frac{1}{\sqrt{|\mathcal{G}_n|}} \tilde{v}_n^{(-g)}, \tag{26}$$

which gives Equation (24). □

B. Data Structure Design and Levels of Vectorization

B.1. ID-to-Index Mapping and Visual Toy Example of Dual CSR

0-Based Indexing. In the raw MovieLens files, user IDs and movie IDs are not consecutive 0-based integers: for example, MovieLens-32M has $N = 84{,}432$ rated movies, but raw movie IDs have gaps and go up to 292,757 (and on the 5M subset, $M = 31{,}205$ users and $N = 48{,}708$ movies also have gaps). Using raw IDs directly as array indices would waste memory on empty rows. When loading the data, we sort the unique user IDs and movie IDs and map each one to a 0-based index:

$$\phi_{\mathcal{U}} : \text{user_id} \mapsto m \in \{0, 1, \dots, M-1\}, \qquad \phi_{\mathcal{I}} : \text{movie_id} \mapsto n \in \{0, 1, \dots, N-1\}, \tag{27}$$

and keep reverse lookup tables $\phi_{\mathcal{U}}^{-1}[m]$ and $\phi_{\mathcal{I}}^{-1}[n]$ so we can get back the original ID and movie title in $O(1)$ time.

Visual Toy Example. Consider a small example with $M = 3$ users, $N = 4$ movies, and $|\Omega| = 6$ observed ratings:

$$R = \begin{pmatrix} 4.0 & \cdot & 5.0 & 3.5 \\ \cdot & 2.0 & \cdot & \cdot \\ 4.5 & \cdot & \cdot & 5.0 \end{pmatrix} \tag{28}$$

User m	user_id	Ω_m	d_m
$m = 0$	10	$\{0, 2, 3\}$	3
$m = 1$	42	$\{1\}$	1
$m = 2$	95	$\{0, 3\}$	2

Movie n	movie_id	Ω^n	d^n
$n = 0$	100	$\{0, 2\}$	2
$n = 1$	350	$\{1\}$	1
$n = 2$	800	$\{0\}$	1
$n = 3$	1200	$\{0, 2\}$	2

Instead of storing the empty entries (·), we build two Compressed Sparse Row (CSR) views in flat arrays. The offset arrays $p^{(u)}$ (length $M + 1$) and $p^{(v)}$ (length $N + 1$) store the starting position of each user or movie so any row can be sliced in $O(1)$ time via $p_m^{(u)} : p_{m+1}^{(u)}$ or $p_n^{(v)} : p_{n+1}^{(v)}$:

1. User-Indexed CSR ($\mathcal{S}_{\text{user}}$) ($p_m^{(u)} : p_{m+1}^{(u)}$, $d_m = p_{m+1}^{(u)} - p_m^{(u)}$):

$$p^{(u)} : \left[\, 0,\ 3,\ 4,\ 6 \,\right]$$

$$c^{(u)} : \Big[\, \underbrace{0,\ 2,\ 3}_{\text{User 0 (0:3)}},\ \underbrace{1}_{\text{User 1 (3:4)}},\ \underbrace{0,\ 3}_{\text{User 2 (4:6)}} \,\Big]$$

$$r^{(u)} : \Big[\, \underbrace{4.0,\ 5.0,\ 3.5}_{\text{User 0}},\ \underbrace{2.0}_{\text{User 1}},\ \underbrace{4.5,\ 5.0}_{\text{User 2}} \,\Big]$$

2. Movie-Indexed CSR ($\mathcal{S}_{\text{movie}}$) ($p_n^{(v)} : p_{n+1}^{(v)}$, $d^n = p_{n+1}^{(v)} - p_n^{(v)}$):

$$p^{(v)} : \left[\, 0,\ 2,\ 3,\ 4,\ 6 \,\right]$$

$$c^{(v)} : \Big[\, \underbrace{0,\ 2}_{\text{Mov 0 (0:2)}},\ \underbrace{1}_{\text{Mov 1 (2:3)}},\ \underbrace{0}_{\text{Mov 2 (3:4)}},\ \underbrace{0,\ 2}_{\text{Mov 3 (4:6)}} \,\Big]$$

$$r^{(v)} : \Big[\, \underbrace{4.0,\ 4.5}_{\text{Mov 0}},\ \underbrace{2.0}_{\text{Mov 1}},\ \underbrace{5.0}_{\text{Mov 2}},\ \underbrace{3.5,\ 5.0}_{\text{Mov 3}} \,\Big]$$

B.2. Four Levels of Vectorization and Parallelization for ALS

Using the CSR layout above, we can compare four ways to compute the bias and trait vector updates (Table 6):

1. **Level 1 (Naive Nested Loops)**: For each user $m \in \{0, \dots, M-1\}$, loop over each rating index $s \in [p_m^{(u)}, p_{m+1}^{(u)})$, read movie $n = c^{(u)}[s]$ and rating $r_{m,n} = r^{(u)}[s]$, and add to the bias residual sum and the $K \times K$ matrix entries one number at a time using nested loops over $i, j \in \{1, \dots, K\}$: $A_m[i,j]$ += $\lambda v_{n,i} v_{n,j}$ and $b_m[i]$ += $\lambda \tilde{r}_{m,n}^{(u)} v_{n,i}$ (and similarly over $\mathcal{S}_{\text{movie}}$ for movies). In an interpreted language, running $|\Omega|K^2$ scalar loop steps (2.56×10^9 steps per half-sweep at $K = 10$ on 32M) takes several minutes per epoch because of slow loop overhead.

2. **Level 2 (Per-User / Per-Movie Matrix Multiplication)**: For a single user m with $d_m = p_{m+1}^{(u)} - p_m^{(u)}$ ratings, slice $\Omega_m = c^{(u)}[p_m^{(u)} : p_{m+1}^{(u)}]$ and collect the rated movie biases $b_{\Omega_m} \in \mathbb{R}^{d_m}$, movie vectors $V_{\Omega_m} \in \mathbb{R}^{d_m \times K}$, and ratings $r_m \in \mathbb{R}^{d_m}$. Both the bias update and the $K \times K$ trait vector update for user m then become matrix-vector and matrix-matrix products:

$$a_m \leftarrow \frac{\lambda \mathbf{1}_{d_m}^\top (r_m - \mu \mathbf{1}_{d_m} - b_{\Omega_m} - V_{\Omega_m} u_m)}{\lambda d_m + \gamma}, \qquad A_m = \lambda V_{\Omega_m}^\top V_{\Omega_m} + \tau I_K, \qquad b_m = \lambda V_{\Omega_m}^\top \tilde{r}_m^{(u)}, \qquad u_m \leftarrow A_m^{-1} b_m. \tag{29}$$

   While this vectorizes the inner sum over Ω_m, calling $M = 200{,}948$ separate small matrix multiplications one after another is still slowed down by function-call overhead because the median user has $d_m = 73$ ratings.

3. **Level 3 (Batched Matrix/Tensor Vectorization Across Many Users)**: Using our contiguous CSR arrays, we can batch many users $\mathcal{B} = \{m_1, \dots, m_B\}$ together by slicing all $|\Omega_{\mathcal{B}}| = p_{m_B+1}^{(u)} - p_{m_1}^{(u)}$ ratings at once, collecting $V_{\mathcal{B}} \in \mathbb{R}^{|\Omega_{\mathcal{B}}| \times K}$, building the 3D outer-product array $\mathcal{T}_{\mathcal{B}} \in \mathbb{R}^{|\Omega_{\mathcal{B}}| \times K \times K}$ with $(\mathcal{T}_{\mathcal{B}})_{s,i,j} = v_{n_s,i} v_{n_s,j}$, summing segments along the first axis using the offsets $p^{(u)}$ to get all B matrices $A_{\mathcal{B}} \in \mathbb{R}^{B \times K \times K}$ and right-hand sides $b_{\mathcal{B}} \in \mathbb{R}^{B \times K}$ at once, and calling a batched solver. However, building $\mathcal{T}_{\mathcal{B}}$ in memory uses $O(|\Omega_{\mathcal{B}}|K^2)$ bytes of temporary RAM ($[0.3, 3.6]$ GB per batch), which slows down training due to memory allocation and cache misses (12.73 s/epoch at $K = 10$ on 5M).

4. **Level 4 (Multi-Core Parallel CSR Loops with In-Place Cholesky)**: To avoid building large 3D arrays, we go back to updating one user or movie at a time as in Levels 1 and 2, compile the loop to machine code, and run the outer loop over users $m \in \{0, \dots, M-1\}$ (or movies $n \in \{0, \dots, N-1\}$) in parallel across T CPU cores. Each thread keeps a small $K \times K$ buffer ($8K^2$ bytes) and a length-K vector ($8K$ bytes) in L1 cache, reads sequentially through $p_m^{(u)} : p_{m+1}^{(u)}$ to fill the lower triangle of $A_m = \lambda V_{\Omega_m}^\top V_{\Omega_m} + \tau I_K$ ($\frac{1}{2}K(K+1)|\Omega|$ multiply-adds), and solves $A_m u_m = b_m$ in-place with Cholesky decomposition ($\frac{1}{6}K^3$ multiply-adds), using under 32 KB of extra RAM across all threads (0.183 s/epoch on 5M; 1.927 s/epoch on 32M at $K = 10$).

C. Complete Recommendation and Polarization Tables

Table 7 reports the complete Top-10 dummy-user recommendations across all three bias-weighting settings (Settings A, B, and C), and Table 8 lists the complete Top-10 most polarizing and Top-10 least polarizing movies with at least 1,000 training ratings on MovieLens-32M.

Table 6. Comparison of asymptotic time per epoch, floating-point operations, and extra working memory across ALS solvers.

Solver Level	Time Complexity / Epoch	Multiply-Adds / Epoch	Extra Working RAM
Bias-Only ALS ($K = 0$)	$O(\|\Omega\| + M + N)$	$2\|\Omega\| + 2(M + N)$	$O(T)$ ($\leq$ 1 KB)
Level 2: Per-Entity MatMul ($K > 0$)	$O(\|\Omega\|K^2 + (M + N)K^3)$	$2\|\Omega\|K^2 + \frac{1}{3}(M + N)K^3$	$O(d_{\max}K)$ ($\leq$ 12 MB)
Level 3: Batched 3D Tensor ($K > 0$)	$O(\|\Omega\|K^2 + (M + N)K^3)$	$2\|\Omega\|K^2 + \frac{1}{3}(M + N)K^3$	$O(\|\Omega_{\text{batch}}\|K^2)$ ([0.3, 3.6] GB)
Level 4: Parallel CSR Cholesky ($K > 0$)	$O\big(\frac{1}{T}(\|\Omega\|K^2 + (M + N)K^3)\big)$	$K(K + 1)\|\Omega\| + \frac{1}{6}(M + N)K^3$	$O(TK^2)$ ($\leq$ 32 KB)
Level 4 + Genre Prior ($K > 0$)	$O\big(\frac{1}{T}(\|\Omega\|K^2 + (M + N)K^3) + G^3\big)$	$K(K + 1)\|\Omega\| + \frac{1}{6}(M + N)K^3$	$O(TK^2 + G^2)$ ($\leq$ 36 KB)

Table 7. Complete Top-10 recommendations for a dummy user who gave 5.0 stars to *The Lord of the Rings: The Fellowship of the Ring (2001)* ($K = 16$ model on MovieLens-32M), comparing three ranking rules $s_\alpha(\text{new}, n) = u_{\text{new}}^\top v_n + \alpha b_n$.

Rank	Recommended Title	Genres	$u_{\text{new}}^\top v_n$	b_n	$\lvert\Omega_{\text{train}}^n\rvert$
Setting A: Full Movie Bias $s_{1.0}(new, n) = u_{new}^\top v_n + b_n$ (All Movies)					
1	Hello & Goodbye (2018)	Documentary	0.0002	+3.2712	1
2	SpongeBob SquarePants: Heroes of Bikini Bottom (2011)	Animation, Comedy	−0.0067	+3.2557	2
3	Wesley (2009)	Biography, Drama	0.0002	+3.1869	1
4	Very Ordinary Couple (2013)	Comedy, Drama, Romance	0.0033	+3.1516	2
5	Raise High the Roof Beam, Carpenters (2018)	Documentary	0.0002	+3.0586	1
Setting B: Downweighted Bias $s_{0.05}(new, n) = u_{new}^\top v_n + 0.05\, b_n$ (All Movies)					
1	**Lord of the Rings: The Return of the King, The (2003)**	Action, Adventure, Fantasy	0.2170	+0.5102	54,037
2	**Lord of the Rings: The Two Towers, The (2002)**	Adventure, Fantasy	0.2143	+0.4784	54,016
3	Hello & Goodbye (2018)	Documentary	0.0002	+3.2712	1
4	Wesley (2009)	Biography, Drama	0.0002	+3.1869	1
5	**Hobbit: An Unexpected Journey, The (2012)**	Adventure, Fantasy, IMAX	0.1590	−0.0259	12,821
Setting C: Downweighted Bias $s_{0.05}(new, n) = u_{new}^\top v_n + 0.05\, b_n$ with $\lvert\Omega_{train}^n\rvert \geq 100$					
1	**Lord of the Rings: The Return of the King, The (2003)**	Action, Adventure, Fantasy	0.2170	+0.5102	54,037
2	**Lord of the Rings: The Two Towers, The (2002)**	Adventure, Fantasy	0.2143	+0.4784	54,016
3	**Hobbit: An Unexpected Journey, The (2012)**	Adventure, Fantasy, IMAX	0.1590	−0.0259	12,821
4	**Hobbit: The Desolation of Smaug, The (2013)**	Adventure, Fantasy, IMAX	0.1550	−0.1144	9,701
5	**The Hobbit: The Battle of the Five Armies (2014)**	Adventure, Fantasy	0.1523	−0.2369	6,951
6	Braveheart (1995)	Action, Drama, War	0.1114	+0.2622	55,834
7	Harry Potter and the Deathly Hallows: Part 2 (2011)	Action, Adventure, Fantasy	0.1110	+0.1150	16,242
8	Harry Potter and the Half-Blood Prince (2009)	Adventure, Fantasy, Mystery	0.1141	+0.0189	16,869
9	Harry Potter and the Order of the Phoenix (2007)	Adventure, Drama, Fantasy	0.1138	+0.0061	16,841
10	Harry Potter and the Prisoner of Azkaban (2004)	Adventure, Fantasy, IMAX	0.1063	+0.1546	25,037

Table 8. Complete Top-10 most polarizing (largest $\|v_n\|_2$) and Top-10 least polarizing (smallest $\|v_n\|_2$) movies with at least 1,000 training ratings ($K = 16$ on MovieLens-32M), along with movie bias b_n, rating count $|\Omega^n_{\text{train}}|$, and rating standard deviation $\sigma(r)$.

Rank	**Movie Title**	$\|v_n\|_2$	**Movie Bias** b_n	$\|\Omega^n_{\text{train}}\|$	**Rating Std** $\sigma(r)$
Top-10 Most Polarizing Movies ($\max \|v_n\|_2$ *among movies with* $\|\Omega^n_{train}\| \geq 1{,}000$*)*					
1	Star Wars: Episode II: Attack of the Clones (2002)	8.000	−0.5765	21,907	1.146
2	Star Wars: Episode III: Revenge of the Sith (2005)	7.819	−0.2665	19,751	1.124
3	Star Wars: Episode I: The Phantom Menace (1999)	7.754	−0.5638	30,338	1.143
4	Dumb & Dumber (1994)	7.732	−0.6289	29,483	1.204
5	Lord of the Rings: The Return of the King, The (2003)	7.730	+0.5102	54,037	1.011
6	Blair Witch Project, The (1999)	7.632	−0.7025	18,180	1.228
7	Lord of the Rings: The Two Towers, The (2002)	7.632	+0.4784	54,016	1.000
8	Star Wars: Episode IV: A New Hope (1977)	7.557	+0.3851	67,968	0.988
9	Harry Potter and the Half-Blood Prince (2009)	7.554	+0.0189	16,869	1.007
10	Lord of the Rings: The Fellowship of the Ring, The (2001)	7.543	+0.5191	58,491	1.008
Top-10 Least Polarizing Movies ($\min \|v_n\|_2$ *among movies with* $\|\Omega^n_{train}\| \geq 1{,}000$*)*					
1	Cobb (1994)	0.892	−0.2659	1,023	0.825
2	Pirates of Silicon Valley (1999)	1.119	+0.1437	1,061	0.778
3	Sea of Love (1989)	1.248	+0.0381	1,038	0.757
4	Black Book (Zwartboek) (2006)	1.263	+0.4203	1,014	0.767
5	Run Silent Run Deep (1958)	1.347	+0.2111	1,089	0.749
6	Wait Until Dark (1967)	1.354	+0.4110	1,132	0.748
7	Proof (2005)	1.366	−0.0011	1,001	0.834
8	Rescue Dawn (2006)	1.371	+0.1189	1,316	0.701
9	Everest (2015)	1.373	−0.0605	1,252	0.823
10	Counterfeiters, The (Die Fälscher) (2007)	1.398	+0.4343	1,131	0.695